

AGILA CATalog App Proposal

By: Niles Cabrera

Background and Rationale

The Ateneans Guided and Inspired by their Love for Animals (AGILA) spearheads the Cat Census (CATalog) project, which seeks to track the population of cats within Ateneo in collaboration with the Trap-Neuter-Vaccinate-Return (TNVR) program. However, this project involves the use of multiple Google Workspace apps that handles internal data acquisition and organization, requiring members to navigate multiple data sheets while strictly adhering to encoding protocols. That opens up **potential data errors** and **makes onboarding new members labor-intensive**. Coupled with the fact that such a system **does not allow for quality-of-life improvements** (i.e. quick look-up, cross-database management, data validation), these issues heavily add to the difficulty of implementing and managing the census.

Furthermore, AGILA's cat adoption/foster system primarily entails the use of a Facebook page for making calls and updates to the public. The problem is that these posts only showcase new or featured cats, and information regarding the rest of the other available cats are buried over time. This **makes it hard for an average person to actually look through and choose adoptable/fosterable cats**—and this may even dissuade them from adopting/fostering from AGILA. Indeed, there exists a list of all such cats in the Google Form used for applications, but it is not readily-available and merely contains their names—it does not have descriptions, pictures, and other relevant information for each of them.

These problems are exactly what a website application can solve. To address both, it can incorporate two interconnected components (internal and external). In a nutshell, the internal side can allow AGILA members to better manage the CATalog through a **centralized dashboard**, while the external side involves hosting an editable and easy-access **list of adoptable/fosterable cats** with a customized display.

It is useful to think of the potential app as an overhead layer over the pre-existing tools and systems used by AGILA—it can be non-disruptive. One benefit of this is that if something does go wrong with it, one can always fall back to such systems. Additionally, in that way it will also preserve the current frameworks and infrastructure for such.

Ultimately, for **minimal cost and required adjustments**, a web app can provide **quality-of-life and systemic improvements** to the current implementation of the CATalog, as well as **better visibility** for the cat adoption/fostering system.

The problem.

AGILA's current CATalog management system uses fragmented tools and data sheets, as well as high friction public interaction channels and processes—leading to considerable manual overhead, error-prone data inputs, missed adoption opportunities, etc.

The solution?

To address the problem, a development team can make a website application that integrates an internal data management system in the form of a dashboard that centralizes the pre-existing tools and processes involved in managing the CATalog (Google Sheets, Google Forms, etc.) as well as an external catalog of adoptable/fosterable cats for public display.

The **dashboard** will model the primary CATalog Google Sheet and cat census workflows in terms of categories as well as data structure and userflow. The following are the specifications for each of its sections:

1. **Overview** - This will display the relevant general statistics of the CATalog. Such will be based on the Home sheet of the CATalog GSheet. It will have a select field that could be used to filter the statistics for a specific location.

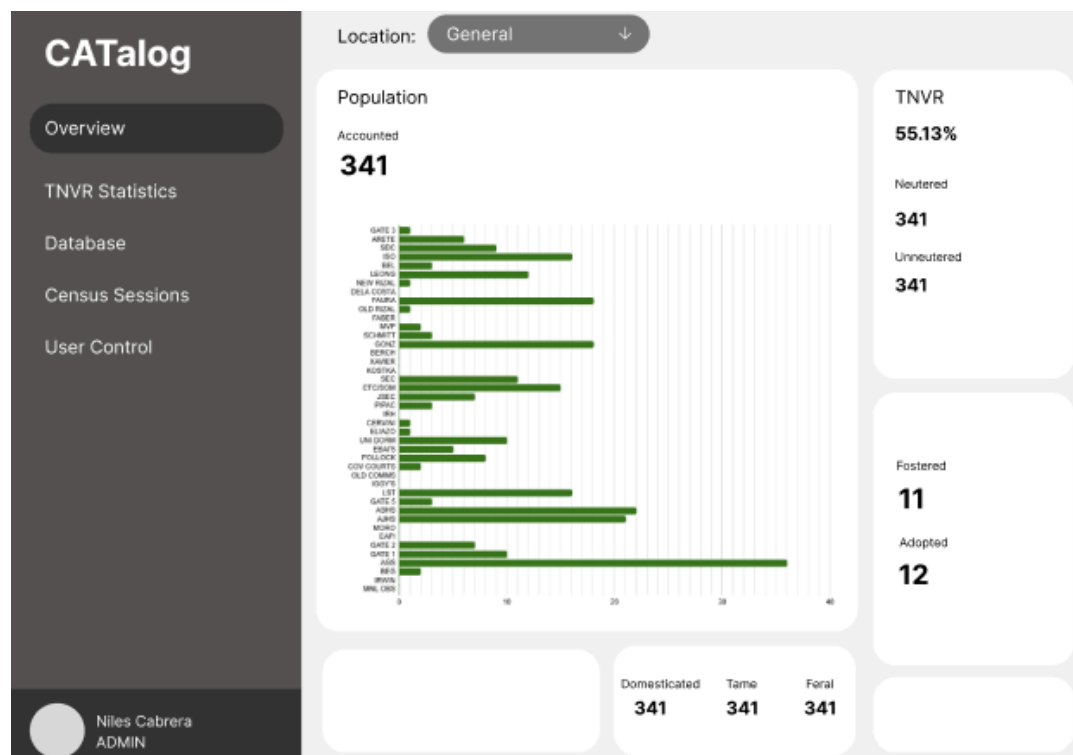


Figure 1: Wireframe of the Dashboard's Overview Section (General).

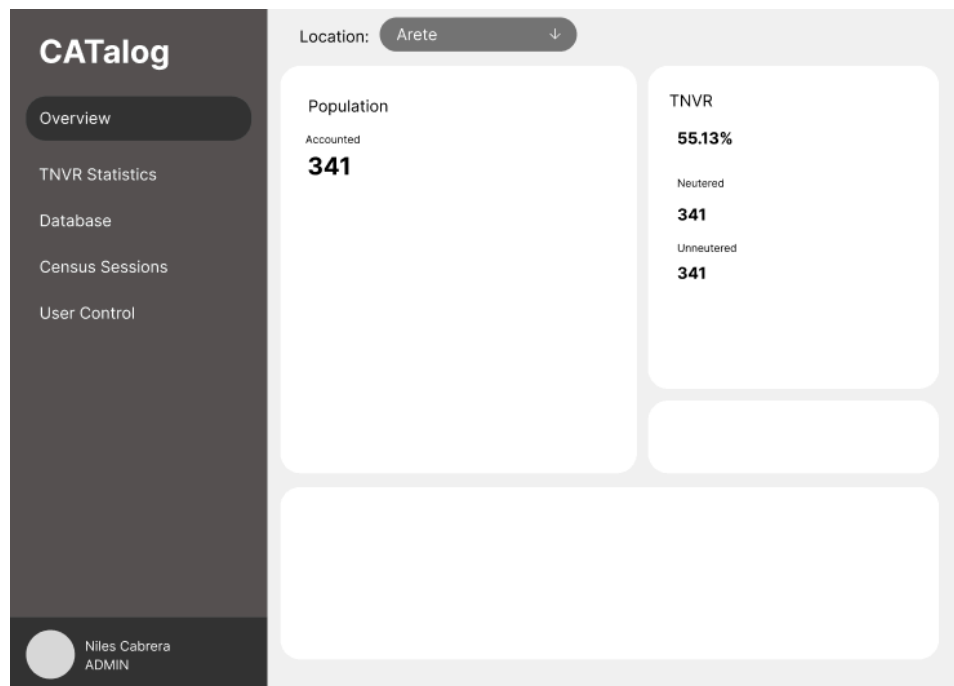


Figure 2: Wireframe of the Dashboard's Overview Section (Filtered by Location)

2. **TNVR Statistics** - This will functionally be similar to the overview section. It will just go more in depth regarding the TNVR statistics especially in terms of gender.

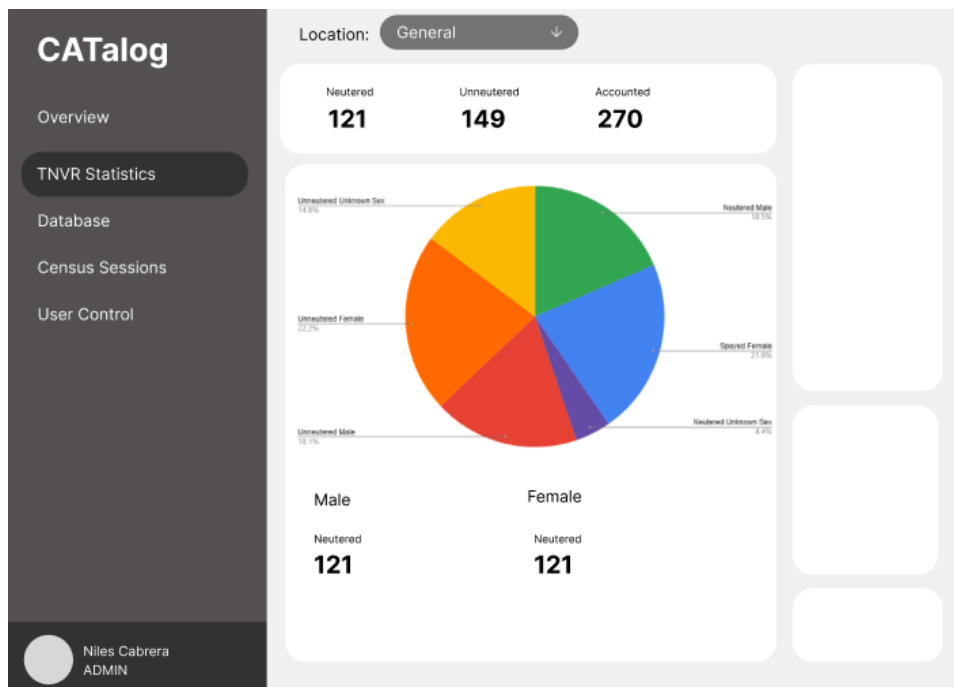


Figure 3: Wireframe of the Dashboard's TNVR Section (General)

3. **Database** - It will list the entire dataset of such that can be navigated with a set of filters and allow CRUD (create, read, update, delete) operations to be performed on its records.

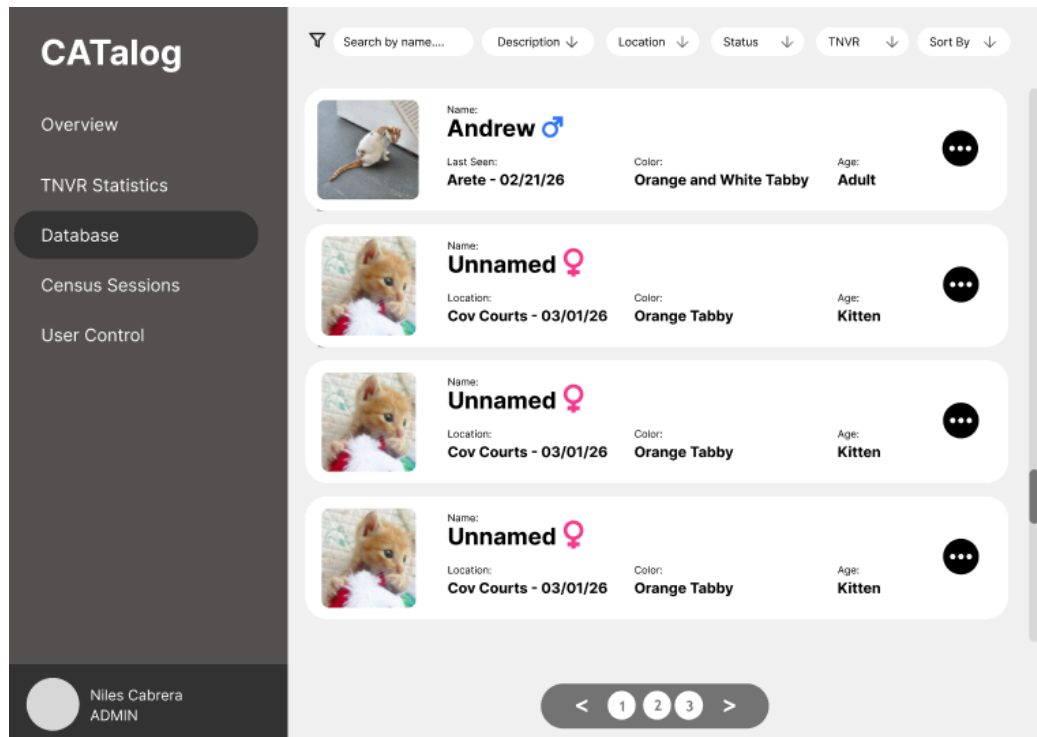


Figure 4: Wireframe of the Dashboard's Database Section

4. **Census Sessions** - The initial landing screen of this will basically be a navigation menu. The first section will list all of the current user's sessions. Under it, there will also be a dropdown section containing a scrollable tracker of various locations and how many days it had been since the last census was held there. The list will be arranged in a descending order of such a count, so that the older ones can be more prioritized in terms of information hierarchy. Beside these sections, there will be the user actions—some of which are only accessible to managers/admins. The Create Session action will refer the user to a saveable form that can be used to input new entries for the CATalog database. Meanwhile, the Review Sessions action will refer the user, if authorized to do so, to a list of all new entries compiled from all of the sessions that have been submitted.

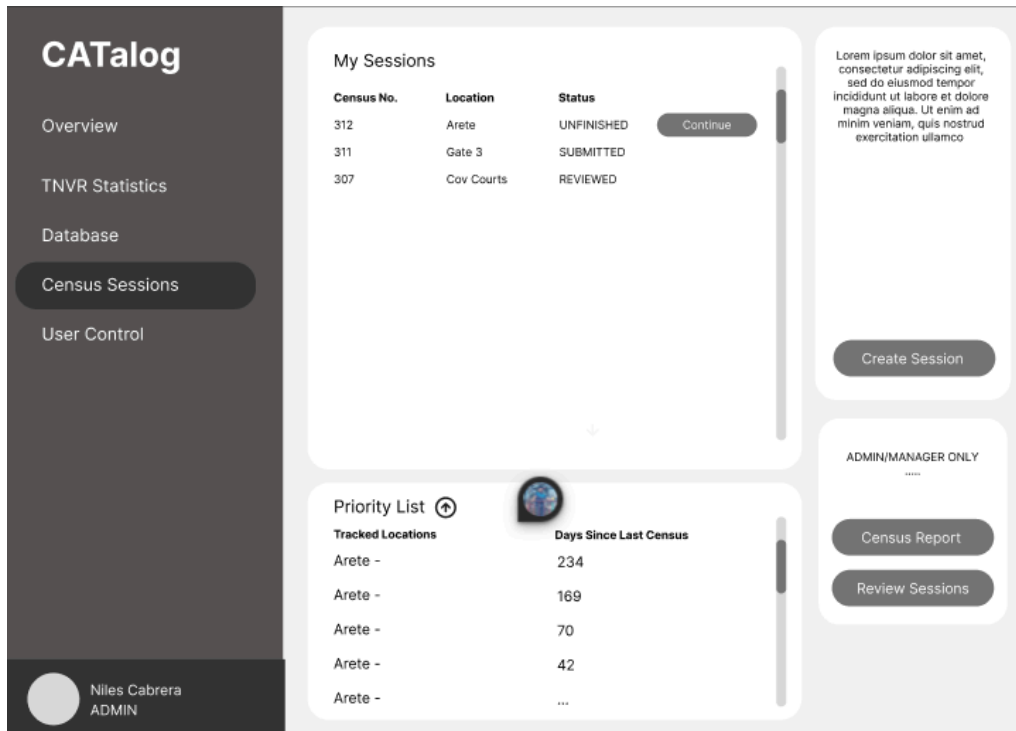


Figure 5: Wireframe of the Dashboard's Census Section (Landing)

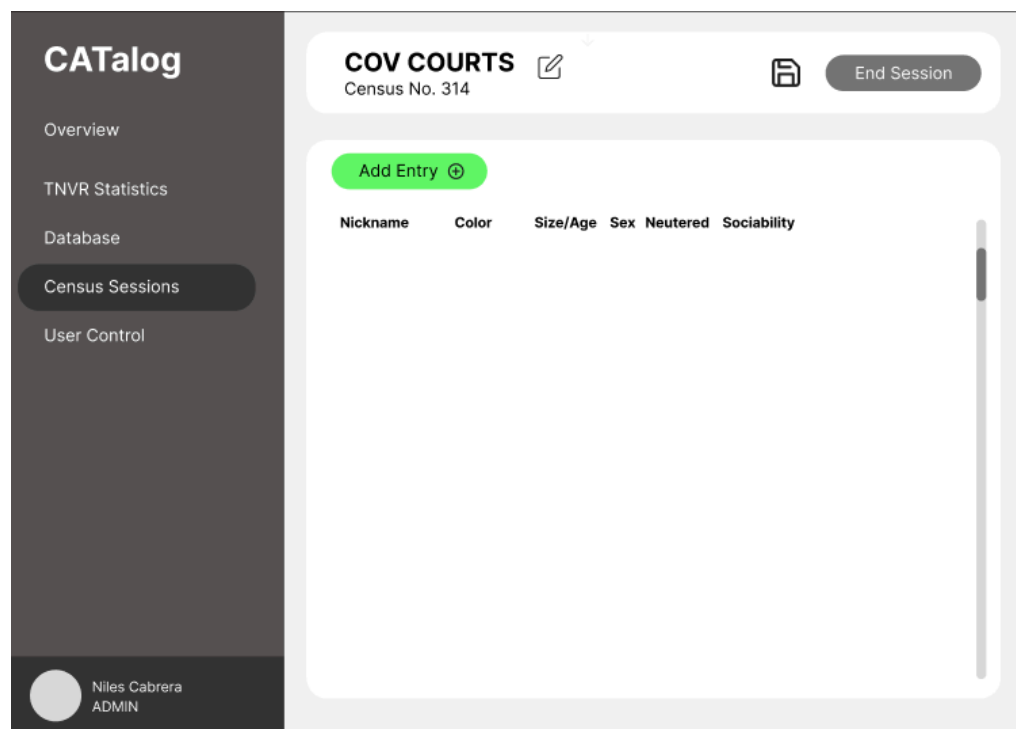


Figure 6: Wireframe of the Dashboard's Census Section (Create Session Form)

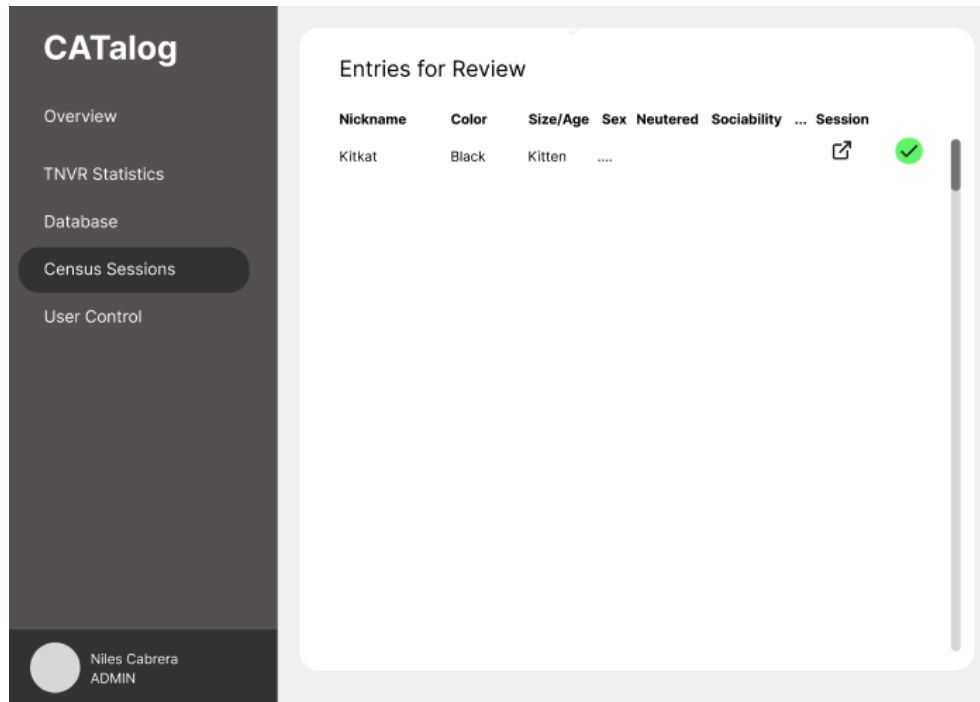


Figure 7: Wireframe of the Dashboard's Census Section (Review Entries)

5. **User Control** - In here, an user with the admin role can add dashboard users and assign/change their roles.

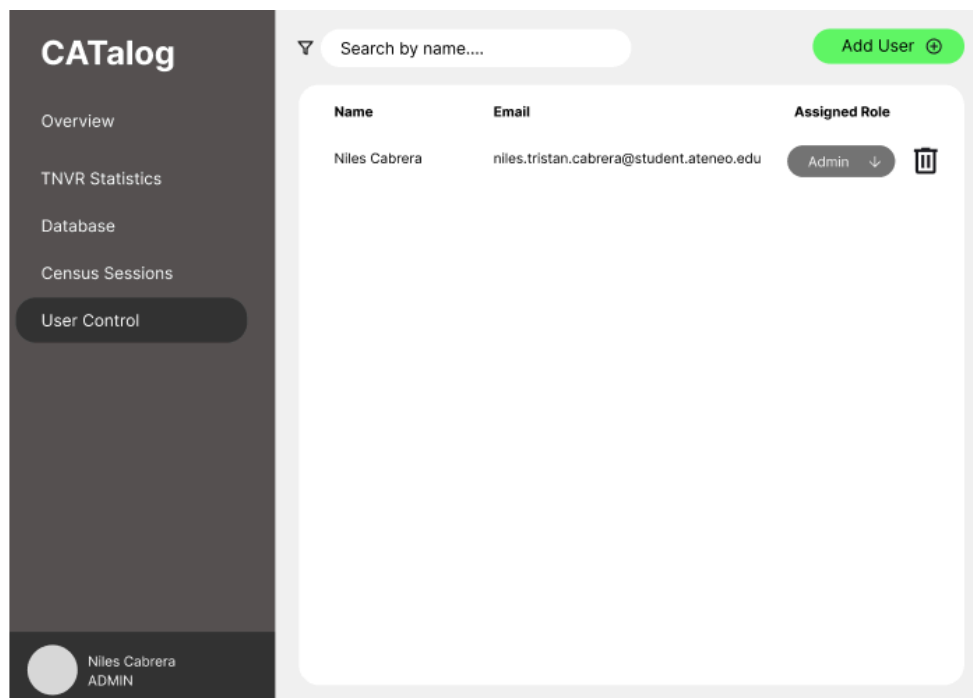


Figure 9: Wireframe of the Dashboard's User Control Section

On the topic of **authentication**, the app will implement a role-based access control (RBAC) system that will basically define what users can see and do depending on their assigned roles. The following is the model that will be used for the app (all user roles have the permissions as those above it):

1. Guest (Unauthenticated) - can access public pages
2. Volunteer - can only read dashboard statistics and database as well as create censuses
3. Manager - can approve census session entries and operate on the database (CRUD)
4. Administrator - can manage user permissions and access levels

Going into more detail, it will integrate a Google OAuth system for automated sign-ins using Google accounts. Such will be configured so that it rejects email addresses that do not end in “@student.ateneo.edu” for an added layer of account security.

Lastly, the **catalog** for adoptable/fosterable cats, which is the main public-facing component of the app, will be connected to the app database—such that any cats listed as adoptable in the dashboard will be put up dynamically.

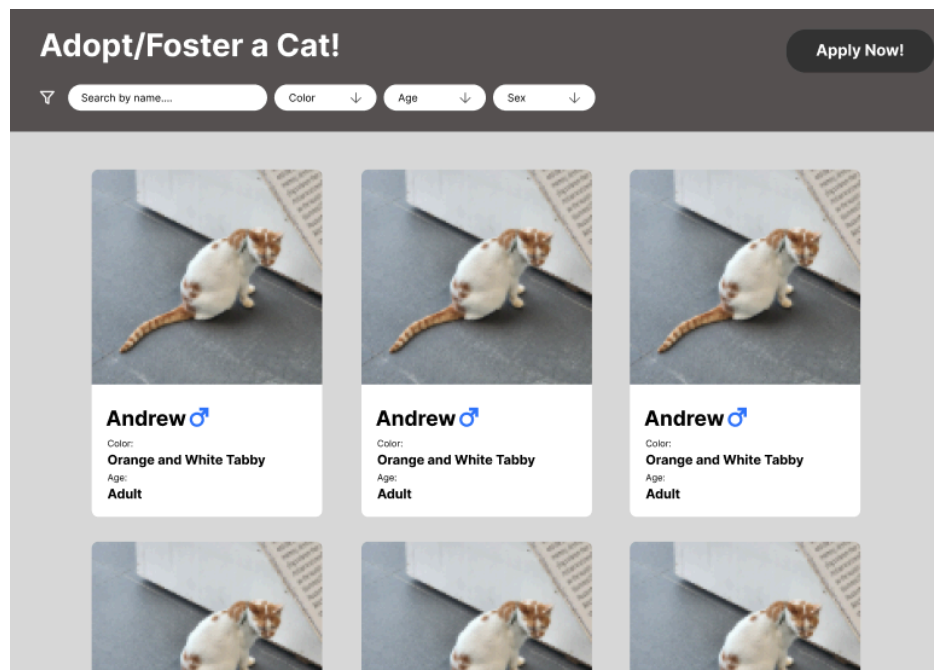


Figure 10: Wireframe of the Public Catalog

How will it be made?

The app will be made through **Next.js**, a modern JavaScript framework for website development, which makes it easily deployable through a hosting service called **Vercel**.

Likewise, **Supabase**, which is a PostgreSQL development platform, will be made to handle and host the app's database, and it will be connected to the codebase using the **Drizzle ORM**.

Shadcn will serve as the primary component library for frontend design, as well as other libraries such as **react-icon**. The use of **TypeScript** and **Zod** will comprise the app's data type safety and validation system. Meanwhile, the **Google Sheets API** will be used for connecting to the pre-established systems of the CATalog, especially in terms of its data sheets (the app database will be considered the main source of truth).

Moreover, the deployed app's link obtained from Vercel can be configured to work under a separate domain name. It can either be deployed as a subdomain of a pre-existing domain name or its own instead through **Cloudflare**, which provides additional cybersecurity and ease-of-use features for managing domains.

Scope

The development team will focus on creating the **MVP (Minimum Viable Product)** version of the app first. Such will cover a functional dashboard with the features listed in the solution section of this proposal and a catalog, ideally with minimal visual design. This is critical for prototyping and optimizing properly for the end product. More importantly, such is also to prevent needlessly overscoping—grounding one's expectations and focus for the app, especially in terms of development time and practicality.

Improvements to the app can be introduced later on. Potential additions may include: an updates/announcements page, a donation system, better visual design, as well as UI/UX (User Experience) optimizations. Other potential implementations will be considered based on future discussions with AGILA.

Furthermore, the app will be expected to serve around 40 internal personnel (managers and volunteers) as well as the general public (visitors), which is projected to be in the hundreds up to a thousand based on AGILA's present social media metrics. **These numbers are well within the scope of the project**, especially in terms of cost minimization and technical design.

Timeline

Development Phase:

Sprint 1: Set-up (Two Weeks: Feb 23 - Mar 8)

1. Project Initiation
2. Authentication System
3. Database Configuration

4. Wireframes

Sprint 2: Systems Implementation (Three weeks: Mar 9 - 29)

1. Dashboard Analytics
2. Database Interaction
3. Census Session System
4. User Control
5. Public Adoption Catalog
6. High-Fidelity Designs
7. Google Sheets API Proof of Concept

Sprint 3: Finalization (Three Weeks: March 30 - April 19)

1. Connecting to Google Sheets systems (API)
2. Final Design Creation and Integration
3. UI/UX Optimizations
4. Bug-Fixing, Security Testing, and Load Testing
5. Documentation Write-Up

Implementation Phase:

Buffer Period (One week: Apr 20 - 26)

Sprint 1: Launch (Three Days: Apr 27 - 29)

1. Project Presentation
2. Onboarding AGILA members for App Adoption
3. Public Announcement and Promotion

Sprint 2: Prototype Testing (One Week: Apr 30 - May 6)

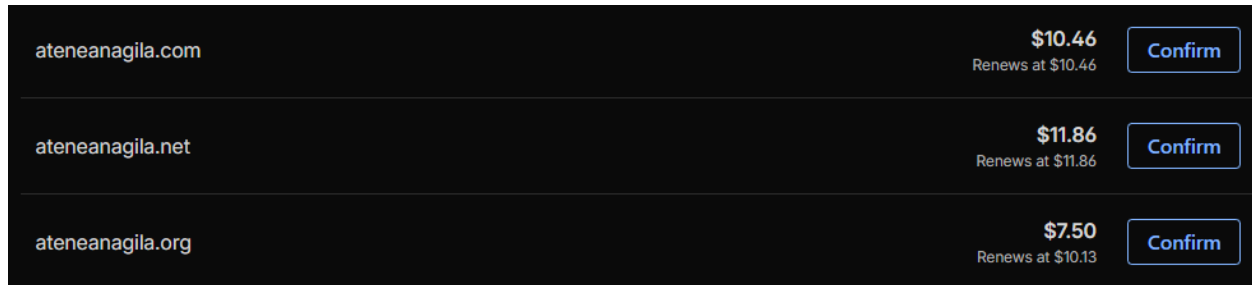
1. Monitoring App Usage
2. Feedback Gathering
3. Optimizing UI/UX and Features

Cost Analysis

Ideally, there can be no cost in developing and maintaining the app. Labor in part of the development team is expected to not be paid for. The hosting services that will be used for the deployment of the app have relatively generous free tiers for usage; and based on its scope, it likely won't need more than such. However, it is important to account for certain considerations that may involve non-mandatory fees.

Primarily, the **domain name** must be considered (website link). This is crucial for the branding aspect of the project. A customized name will give the app an official and credible appearance—making it appealing and easier to use for users. It also helps in terms of Search Engine Optimization (SEO) or the searchability of the website in typical browsers. Now, how exactly does one make a custom domain name? Well, Vercel provides a default name such as

“appname.vercel.app” (free), however that can be changed by connecting it to another domain. If so, the team has to **buy a domain** from a registrar like Cloudflare. The following picture can be used for price reference:



The image shows a screenshot of the Cloudflare domain registrar interface. It displays three domain options: ateneanagila.com, ateneanagila.net, and ateneanagila.org. Each option shows its price and a 'Confirm' button. The prices are \$10.46 for .com, \$11.86 for .net, and \$7.50 for .org. Below each price, it says 'Renews at' followed by the same price.

ateneanagila.com	\$10.46 Renews at \$10.46	Confirm
ateneanagila.net	\$11.86 Renews at \$11.86	Confirm
ateneanagila.org	\$7.50 Renews at \$10.13	Confirm

Figure 11: Actual Price Reference from the Cloudflare Registrar

Note that the name to be chosen is still up to discussion, and one can check available domain names through the Cloudflare dashboard. For such, a recommendation of the team would be “ateneanAGILA.org” for it is appropriate for the app and only costs **\$7.50 or Php 433.29** (as of current exchange rates) to have **annually**.

Additionally, as aforementioned in the first paragraph, the free tiers should be enough for the app—however one should also consider the possibility of upgrading from such tiers for the sake of transparency and potential scalability. This will primarily only be necessary if the app will be expecting a large number of users, integrating complex systems, or requiring technical features (in terms of cybersecurity and optimization). For specific details regarding such, visit <https://supabase.com/pricing> for Supabase, <https://vercel.com/pricing> for Vercel, and <https://www.cloudflare.com/lp/pg-all-plans-ssl> for Cloudflare.

Potential Concerns

- 1. Project Handover** - The initial development team, composed of Ateneo undergrad students, can only maintain and manage the project for a definite period of time. As such, certain measures will be taken in order to ensure that such responsibilities can be passed on to future developers. First and foremost, a documentation write-up will be made going over the codebase (structure, functions, etc.), account management, design decisions, and other relevant information about the app. Credentials for hosting and deployment services will be registered under an AGILA-controlled Google account. Access will be managed through shared ownership (e.g., multiple AGILA officers added as administrators), rather than relying on a single custodian. Credentials will be stored in a secure password manager accessible to designated officers (BitWarden). Moreover, the team will be open to leading future onboarding and teaching sessions about this matter.

2. **Cybersecurity** - While the application is not expected to be a high-value target for cybersecurity threats, it will still adhere to standard web security protocols to mitigate automated and opportunistic threats. As such, the team will be following conventional security practices in the codebase implementation, account management, and minimization of attack surfaces. The app is not going to be handling sensitive data in the form of transaction details involved in the donations and other financial processes of AGILA. It will also use modern frameworks and systems involved with web design, such as Cloudflare services and Next.js, as they are constantly updated and managed by professional software engineers independently.
3. **Development Team** - For a project of this scale, one would need at most a team of three people with their own respective roles, namely a **Frontend Developer, Backend Developer, and a UI/UX Designer**. There are already certain candidates for consideration for such. Niles Cabrera, the one who made this initiative, will act as the overall project manager as well as the Backend Developer. The other roles will be filled out accordingly with individuals expressly acknowledged by AGILA after this proposal is approved. Going forward, technical decisions will also be documented and reviewed with **AGILA officers** to ensure transparency and organizational alignment.
4. **Failure Cases** - The development team will handle all system failures. Identified below are some of the failure cases that are most likely to happen—and how exactly they'll be addressed:
 - A. **Hosting or Platform Outage**

In the event of temporary downtime from hosting providers, AGILA may revert to its existing Google Sheets-based workflows. Since the application is designed as an overlay layer, there will always be a fallback system for such. The database will be reconciled with the Google Sheet data (with new potential changes) once the app is functional again.
 - B. **Database Corruption or Data Loss**

Automated database backups will be enabled through Supabase. Periodic exports may also be maintained for redundancy.
 - C. **Authentication Misconfiguration**

Multiple administrators will retain access to admin permissions to prevent lockout scenarios. The codebase can also be changed by the developers accordingly as a last resort.

5. **Website Design Collaboration** - The creatives department of AGILA can work with the UI/UX Designer and Frontend Developer of the team in order to make the visual design of the app. As the team will be using **Figma**, all parties can collaborate seamlessly through comments and sharing designs in a managed Figma Design File (coupled with other communication channels). A primary concern, however, is that AGILA's personnel may not be knowledgeable in creating designs that are actually necessary and feasibly translatable into code for the website (they have to consider limitations too). To bridge this potential gap, the development team will have to ensure constant communication with them, as well as establish plans and guidelines accordingly.

Expected Outcomes

- A fully functional and deployable web app
- Improved efficiency in the CATalog workflows/processes
- Reduced manual workload and data errors
- Positive user feedback and adoption within CATalog personnel
- Significantly increased adoption rates
- Scalable architecture for future feature expansion